# L'Architecture Client-Serveur et le Protocole HTTP

**Thème principal :** Introduction aux Technologies Web et Protocoles de Communication

**Synthèse exécutive :** Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvrira comment un navigateur (client) formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP (2xx, 3xx, 4xx, 5xx) pour diagnostiquer et déboguer efficacement des applications web.

## L'Architecture Client-Serveur et le Principe de Communication

L'architecture client-serveur est le modèle de communication fondamental du Web. Contrairement à un réseau pair-à-pair (P2P) où chaque machine est à la fois cliente et serveuse, ce modèle établit une distinction stricte des rôles. Le client (le navigateur web de l'utilisateur, comme Firefox, Chrome ou Safari) initie toujours la communication en envoyant une requête HTTP à travers le réseau. Le serveur web (comme Apache, Nginx ou IIS), quant à lui, reste dans un état d'écoute passive. Dès qu'il reçoit une requête, il la traite, accède éventuellement à des ressources locales ou à une base de données, puis renvoie une réponse HTTP contenant la ressource demandée (HTML, CSS, image, JSON) ou un message d'erreur.

- Client (Navigateur) : Logiciel actif qui prend l'initiative d'envoyer une requête pour obtenir une ressource.
- Serveur Web : Machine ou processus logiciel passif qui écoute sur un port spécifique (ex: 80 pour HTTP, 443 pour HTTPS) et traite les requêtes entrantes.
- Communication Asynchrone : Le client initie l'échange, le serveur traite la demande puis renvoie la réponse, mettant fin à la transaction.

***Cas concret enseignant :*** *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé afin de reconstituer la page complète.*

## Le Protocole HTTP et sa Nature Stateless (Sans État)

Une caractéristique essentielle du protocole HTTP est qu'il est 'stateless' (sans état). Cela signifie que le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client. Chaque transaction requête-réponse est isolée et autonome. Si cette absence d'état simplifie grandement l'architecture et améliore la scalabilité des serveurs (qui n'ont pas à maintenir des millions de connexions ouvertes), elle pose un problème pour les applications modernes qui nécessitent un suivi de l'utilisateur (comme le maintien d'une session de connexion ou la gestion d'un panier d'achat). Pour pallier cette limite, des mécanismes complémentaires comme les cookies (stockés sur le client) et les sessions (gérées côté serveur) ont été introduits pour simuler un état persistant.

- Protocole Stateless (Sans état) : Caractéristique fondamentale de HTTP où chaque échange requête-réponse est indépendant, sans mémoire des interactions passées.
- Mécanismes de Session : Utilisation de cookies et de sessions côté serveur pour pallier l'absence d'état native de HTTP.

***Cas concret enseignant :*** *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase.*

## Structure des Messages HTTP : Requêtes et Réponses

Une requête HTTP est un message textuel structuré de la manière suivante : 1. La ligne de requête : Elle contient la méthode (le verbe HTTP), l'URI (le chemin de la ressource) et la version du protocole (ex: HTTP/1.1). 2. Les

en-têtes (Headers) : Des lignes de métadonnées fournissant des informations contextuelles (ex: 'Host' pour spécifier le domaine cible, 'User-Agent' pour identifier le navigateur). 3. Une ligne vide : Elle sépare obligatoirement les en-têtes du corps. 4. Le corps (Body) : Optionnel, il contient les données transmises (utilisé principalement avec POST). La réponse du serveur adopte une structure similaire, mais commence par une 'ligne de statut' contenant la version du protocole, un code de statut numérique à trois chiffres et une phrase d'explication textuelle (ex: HTTP/1.1 200 OK).

- Ligne de requête : Contient la méthode (GET, POST), l'URI de la ressource et la version du protocole.
- En-têtes (Headers) : Métadonnées sous forme de clés-valeurs décrivant le client, le serveur ou le contenu de la transaction.
- Corps (Body) : Zone optionnelle contenant les données brutes envoyées au serveur (ex: données de formulaire, JSON).

***Cas concret enseignant :*** *Le professeur insiste sur le fait que le protocole HTTP est purement textuel : on peut littéralement lire les requêtes et réponses en clair si elles ne sont pas chiffrées par HTTPS. C'est pourquoi l'utilisation de HTTP non sécurisé sur un réseau public (comme un Wi-Fi public) expose toutes les données en clair à n'importe quel attaquant équipé d'un sniffer de paquets.*

## Les Méthodes HTTP Fondamentales : GET vs POST

Le choix de la méthode HTTP dépend de la nature de l'action à réaliser. La méthode GET est conçue pour la simple consultation. Comme ses paramètres sont intégrés directement dans l'URL (ex: ?search=informatique), elle présente des limites de taille et de sécurité (les paramètres apparaissent dans l'historique du navigateur et les logs du serveur). À l'inverse, la méthode POST est conçue pour soumettre des données qui vont altérer l'état du système (comme l'insertion d'une ligne en base de données). Les données sont encapsulées dans le corps (body) de la requête, ce qui permet de transférer de grands volumes d'informations de manière invisible dans l'URL, bien que cela ne dispense pas du chiffrement HTTPS pour garantir la confidentialité.

- Méthode GET : Utilisée pour demander/récupérer une ressource sans modifier l'état du serveur (méthode dite 'sûre' et 'idempotente'). Les paramètres passent directement dans l'URL.
- Méthode POST : Utilisée pour soumettre des données destinées à modifier l'état du serveur (création d'un compte, envoi d'un formulaire). Les données transitent dans le corps de la requête.

***Cas concret enseignant :*** *Le professeur explique que l'utilisation de GET pour des données sensibles comme un mot de passe est une faille de sécurité majeure visible directement dans l'historique du navigateur ou sur l'écran de n'importe qui passant derrière l'utilisateur (shoulder surfing).*

## La Classification des Codes de Statut HTTP

Chaque réponse HTTP renvoyée par le serveur commence par une ligne de statut contenant un code numérique à trois chiffres. Ce code permet au client de comprendre instantanément le résultat de sa demande sans avoir à analyser l'intégralité du contenu. Il est crucial pour un développeur de faire la distinction entre les erreurs de la famille 4xx et 5xx. Une erreur 4xx signifie que le problème vient du client (URL mal orthographiée, absence d'autorisation, ressource inexistante). Le développeur doit alors corriger le comportement de son application cliente ou l'URL saisie. Une erreur 5xx signifie que le client a fait une demande correcte, mais que le serveur a rencontré un problème interne (crash de script, base de données inaccessible, timeout). Le développeur doit alors intervenir directement sur le code ou la configuration du serveur.

- Codes 2xx (Succès) : Indiquent que la requête a été reçue, comprise et acceptée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Indiquent que des actions supplémentaires sont requises pour compléter la requête (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Indiquent une anomalie dans la requête formulée par le client (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Indiquent que le serveur a échoué à traiter une requête pourtant valide (ex: 500 Internal Server Error).

***Cas concret enseignant :*** *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
**Réponse du professeur :** Pour un formulaire de connexion, il faut impérativement utiliser POST. Si vous utilisez GET, les identifiants et mots de passe saisis vont être concaténés à la fin de l'URL (ex: login.php?user=matei&password;=123). Cela signifie que le mot de passe apparaîtra en clair sur l'écran, sera enregistré dans l'historique de navigation du navigateur, et sera stocké en clair dans les fichiers de log du serveur web. Avec POST, les données transitent de manière invisible dans le corps de la requête, ce qui est beaucoup plus propre et sécurisé (à condition d'utiliser HTTPS pour chiffrer le canal).

## ■ Exercice & Démonstration d'Application

**Énoncé :** Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

**Résolution pas à pas :**
1. Analyse de la ligne de requête : La première ligne est 'GET /images/logo.png HTTP/1.1'. Le premier mot est le verbe ou la méthode HTTP, ici 'GET', qui indique une demande de récupération de ressource sans modification d'état. La ressource demandée (URI) est '/images/logo.png'. La version du protocole utilisée est 'HTTP/1.1'. 2. Identification de l'hôte : L'en-tête 'Host: iutbayonne.univ-pau.fr' spécifie le nom de domaine du serveur auquel la requête est adressée. C'est indispensable pour les serveurs hébergeant plusieurs sites web sur la même adresse IP (virtual hosting). 3. Analyse de l'agent utilisateur : L'en-tête 'User-Agent: Mozilla/5.0' indique au serveur quel type de navigateur ou de client effectue la requête. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas dans le répertoire '/images/' du serveur web, le serveur ne peut pas traiter la demande avec succès. Il doit générer une réponse d'erreur imputable au client (qui a demandé une mauvaise URL). Selon la spécification HTTP, le serveur renverra un code de statut de la classe 4xx, spécifiquement le code '404 Not Found', généralement accompagné d'un corps de réponse contenant une page HTML d'erreur conviviale.

**Résultat : Méthode : GET Ressource : /images/logo.png Hôte cible : iutbayonne.univ-pau.fr Code de retour si absent : 404 Not Found (Erreur Client)**

## ■ Auto-Évaluation Flash d'Examen

**Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?**

*R : C'est un protocole sans mémoire : chaque couple requête-réponse est traité de manière totalement indépendante. Le serveur oublie l'identité et les actions précédentes du client dès que la réponse est envoyée.*

**Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?**

*R : Les codes 4xx désignent des erreurs côté client (ex: mauvaise URL avec le code 404), tandis que les codes 5xx désignent des erreurs internes côté serveur (ex: crash de script ou base de données inaccessible avec le code 500).*

**Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?**

*R : Parce que la méthode GET expose les paramètres directement dans l'URL, ce qui rend les données sensibles visibles à l'écran, dans l'historique du navigateur et dans les logs du serveur.*